VINUNIVERSITY

Hạ tầng đám mây cho AI

AICB-P2T2 · Ngày 16 · Chương 4 — Hạ tầng

Giảng viên Nguyễn Hải Dương

VinUniversity · Phase 2 · Track 2 · Tuần 4

HÃY SUY NGHĨ...

“Một startup chi 50.000 USD mỗi tháng cho bộ xử lý đồ hoạ (GPU) và giảm còn 12.000 USD nhờ chọn đúng cấu hình máy và dùng máy ảo giá rẻ. Với hệ thống AI của chúng ta, nên thuê đám mây (cloud) hay tự vận hành?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

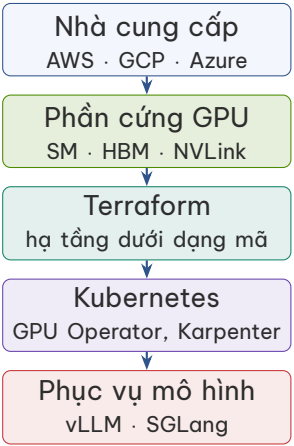
Nội dung bài học

1. Nhà cung cấp đám mây cho AI
2. Nền tảng đám mây (IaaS, PaaS, AI-aaS)
3. Bên trong GPU và phần cứng AI
4. Phiên bản GPU và chi phí
5. Terraform — hạ tầng dưới dạng mã
6. Docker và Kubernetes cho AI
7. Mạng và lưu trữ
8. Hạ tầng agent: 8 lớp
9. Bộ phục vụ mô hình (vLLM, SGLang)
10. Bức tranh đám mây và hạ tầng AI 2026

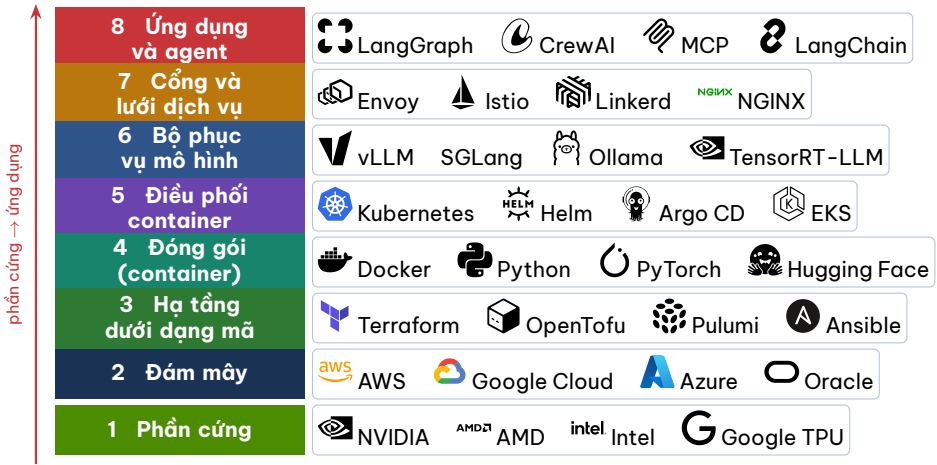
Mục tiêu bài học

Sau bài học này, học viên có thể:

1. Chọn nhà cung cấp đám mây phù hợp với khối lượng công việc AI cụ thể
2. Đọc cấu tạo GPU và chọn đúng loại GPU, đúng dung lượng bộ nhớ GPU (VRAM)
3. Thiết kế môi trường GPU tối ưu chi phí, mô tả bằng Terraform
4. Triển khai (deployment) bộ phục vụ mô hình (serving engine) trên Kubernetes thành điểm cuối (endpoint) dùng được ở môi trường vận hành (production)



Bản đồ bộ công nghệ (stack) hạ tầng AI



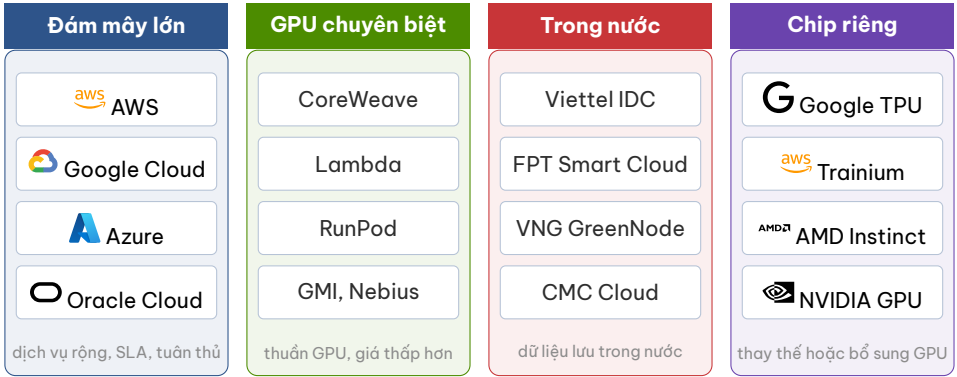
Bài học đi từ **dưới lên**: phần cứng → đám mây → mô tả hạ tầng bằng mã → điều phối (orchestration) → phục vụ mô hình.

01

Nhà cung cấp đám mây cho AI

Ba đám mây lớn, đám mây GPU chuyên biệt, đám mây trong nước và cách chọn

Bản đồ nhà cung cấp hạ tầng AI



Chọn dựa trên bốn yếu tố: **loại khối lượng** × **ngân sách** × **tuân thủ** × **độ trễ (latency)**.

Ba đám mây lớn cho khối lượng AI

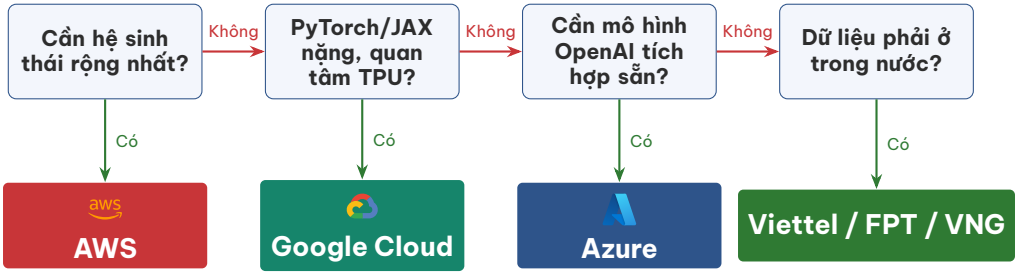
	GPU và chip chủ lực	Dịch vụ AI	Nên chọn khi
AWS	p5 (H100), p5en (H200), p6-b200, p6-b300, UltraServer GB200/GB300; chip Trainium	Bedrock, SageMaker HyperPod, EKS	Cần hệ sinh thái rộng nhất, yêu cầu tuân thủ doanh nghiệp
Google Cloud	A3 Mega (H100), A3 Ultra (H200), A4 (B200), A4X (GB200); TPU Ironwood	Vertex AI, GKE	Huấn luyện nhiều bằng JAX/PyTorch, quan tâm TPU, cụm GKE
Azure	ND H100 v5, ND H200 v5, ND GB200 v6, ND GB300 v6	Azure OpenAI / Foundry, Azure ML	Dùng hệ Microsoft, cần mô hình OpenAI tích hợp sẵn

Điểm chung: hạ tầng mạng và an ninh trưởng thành, vùng khả dụng nhiều, hỗ trợ doanh nghiệp.

Điểm khác: giá niêm yết GPU cao hơn đám mây chuyên biệt 1,5 đến 3 lần (xem chương 4); đổi lại có dịch vụ quản lý.

Nguồn: tài liệu EC2, Compute Engine và Azure GPU sizes (01/10/2026)

Cây quyết định chọn đám mây



Không rơi vào nhánh nào? Nếu chỉ cần GPU giá thấp và đội có kỹ năng hạ tầng: xét đám mây GPU chuyên biệt (CoreWeave, Lambda, RunPod).

Đám mây trong nước đã có GPU thế hệ mới

Nhà cung cấp	GPU công bố
FPT AI Factory	HGX H100, H200; cụm HGX B300 truy cập sớm (từ 2,19 USD/giờ); trung tâm tại Việt Nam và Nhật Bản
Viettel Cloud AI-Native	H200 tại trung tâm dữ liệu Hoà Lạc 2 (Uptime Tier III), kèm DGX B200
VNG GreenNode	HGX H100, L40S, A40 với InfiniBand
CMC	kế hoạch 1 000 GPU GH200 (công bố 2024), chưa xác minh tình trạng 2026

Khung pháp lý hiện hành

- Luật Dữ liệu 60/2024: hiệu lực 01/07/2025
- Luật Bảo vệ dữ liệu cá nhân 91/2025: hiệu lực 01/01/2026; Nghị định 356/2025 thay Nghị định 13/2023
- Luật Trí tuệ nhân tạo 134/2025: áp dụng từ 01/03/2026

Lưu ý: Thông tin cũ cho rằng đám mây trong nước "chỉ có T4/V100" đã lỗi thời. Hạn chế còn lại: ít vùng khả dụng và ít dịch vụ quản lý hơn đám mây lớn.

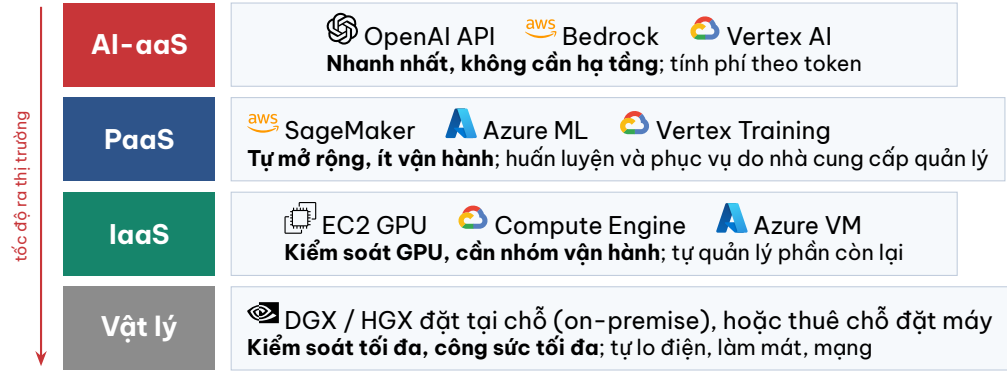
Nguồn: FPT AI Factory; VnExpress (31/07/2026); VNG vServer; EY Vietnam (pháp lý). Số GPU thực tế: hỏi nhà cung cấp

Nền tảng đám mây cho AI

IaaS, PaaS, AI-aaS, trách nhiệm chia sẻ và vùng khởi tạo chuẩn (landing zone)

02

Bốn mô hình triển khai AI: từ API đến phần cứng



Hệ thống AI thường là **kết hợp**: tạo mẫu bằng AI-aaS → phục vụ bằng PaaS → huấn luyện nặng bằng IaaS.

Trách nhiệm chia sẻ (shared responsibility)

Nhà cung cấp chịu trách nhiệm

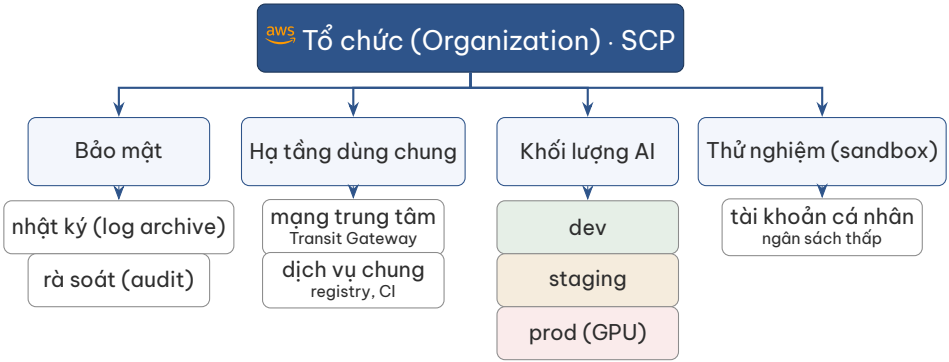
- Hạ tầng vật lý và lớp ảo hoá
- Mạng lõi và chống tấn công từ chối dịch vụ (DDoS)
- Bảo trì phần cứng, độ sẵn sàng của vùng

Nhóm vận hành chịu trách nhiệm

- Bảo mật mô hình và mã hoá dữ liệu
- Kiểm soát truy cập theo quyền tối thiểu (least privilege)
- Chống tấn công chèn lệnh (prompt injection)
- Tuân thủ luật bảo vệ dữ liệu cá nhân (GDPR, luật Việt Nam)

Sự cố phổ biến nhất nằm ở phía người dùng: kho lưu trữ (bucket) để công khai, khoá truy cập lộ trong mã nguồn, quyền `*:*` cấp cho mọi vai trò.

Vùng khởi tạo chuẩn (landing zone)



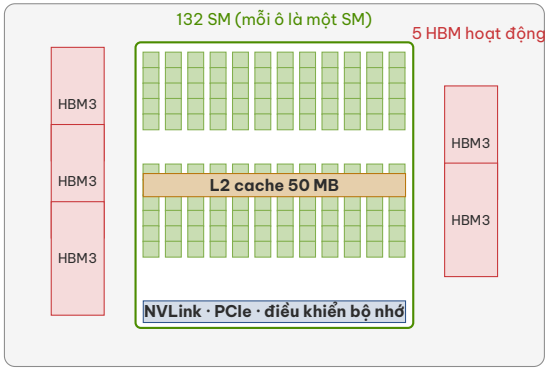
Rào chắn: SCP chặn vùng ngoài danh sách cho phép · bắt buộc mã hoá · cấm tạo bucket công khai · giới hạn loại máy GPU đắt

Sai sót ở bước thiết kế ban đầu rất tốn kém để sửa: gộp mọi thứ trong một tài khoản khiến không thể tách ngân sách, quyền và nhật ký (log) về sau.

Bên trong GPU và phần cứng AI

Từ lõi tính toán, bộ nhớ băng thông cao (HBM), liên kết NVLink đến rack và trung tâm dữ liệu

Giải phẫu một GPU trung tâm dữ liệu: ví dụ H100



Các khối chính

- **SM** (cụm đa xử lý luồng, streaming multiprocessor): đơn vị tính toán cơ bản; H100 có 132 SM
- **Bộ nhớ đệm L2** (L2 cache) 50 MB dùng chung cho mọi SM
- **HBM3** 80 GB nằm cạnh die, qua bus 5 120 bit
- **NVLink** nối GPU với GPU; **PCIe** nối với CPU

Sơ đồ minh hoạ bố cục, không đúng tỉ lệ thật. 80 tỉ transistor, 814 mm^2, tiến trình TSMC 4N.

Nguồn: NVIDIA, Hopper Architecture In-Depth

Sơ đồ chip GH100 đầy đủ của NVIDIA

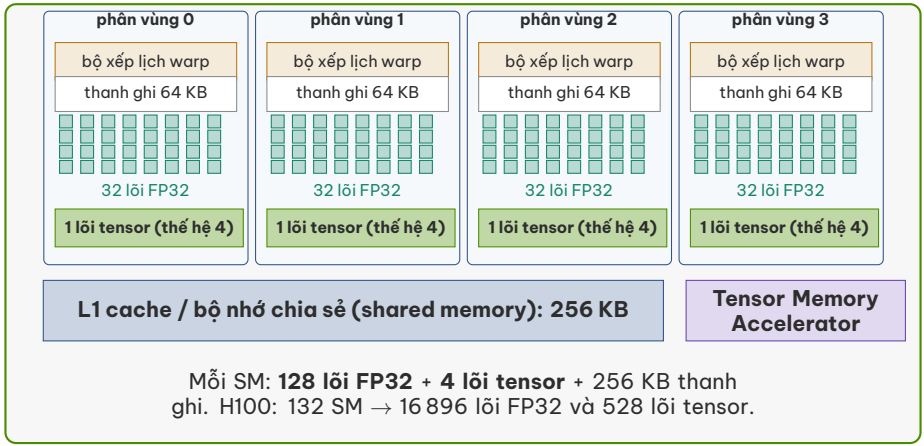


8 cụm GPC chứa các SM (144 SM; H100 bật 132) · **dải xanh giữa**: bộ nhớ đệm L2 dùng chung · **hai bên**: 6 khối HBM3 (H100 SXM5 dùng 5) · **trên**: PCIe 5.0 · **dưới**: 18 cổng NVLink.

Nguồn: NVIDIA, Hopper Architecture In-Depth

Bên trong một SM của GPU

1 SM của H100 (132 SM mỗi GPU)



Nguồn: NVIDIA, Hopper Architecture In-Depth (Bảng 3); sơ đồ vẽ lại, đơn giản hoá

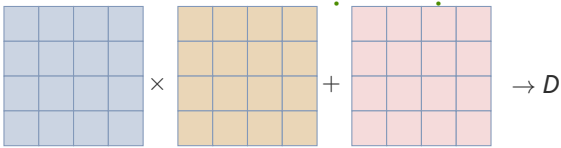
Lõi CUDA và lõi tensor khác nhau thế nào?

Lõi CUDA: một phép tính mỗi lần

$$d = a \times b + c$$

một số nhân với một số, cộng một số

Lõi tensor: cả khối ma trận mỗi lệnh



nhân hai khối nhỏ rồi cộng dồn (MMA: matrix multiply-accumulate); đầu vào độ chính xác thấp, tích luỹ độ chính xác cao

Vì sao lõi tensor nhanh hơn

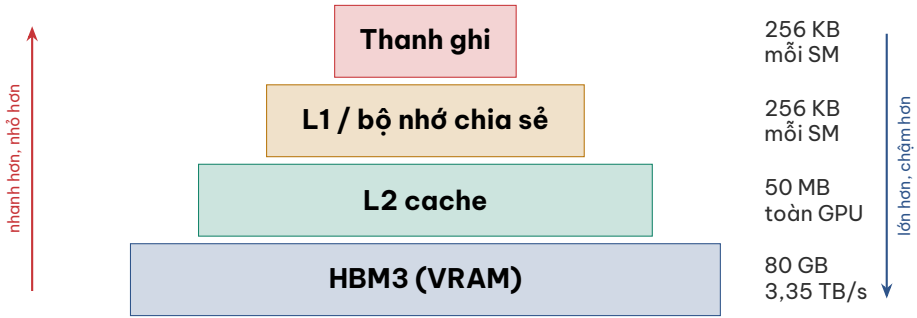
Một lệnh làm hàng trăm phép nhân-cộng, đọc dữ liệu từ thanh ghi một lần. Lõi CUDA phải đọc, tính, ghi cho từng phép.

Hệ quả

Thông lượng (throughput) suy luận của LLM gần như hoàn toàn đến từ lõi tensor; số lõi CUDA ít ảnh hưởng. Muốn dùng được, phép tính phải ở các định dạng số mà lõi tensor hỗ trợ.

Nguồn: NVIDIA, Hopper Architecture In-Depth

Phân cấp bộ nhớ trong GPU: nhỏ thì nhanh, lớn thì chậm

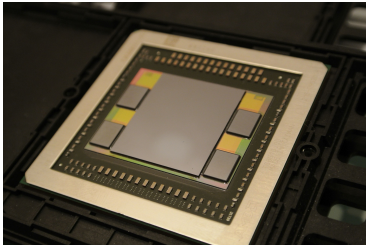


Chương trình GPU nhanh khi dữ liệu được **dùng lại** nhiều lần ở tầng trên (tile trong bộ nhớ chia sẻ) trước khi quay về HBM.

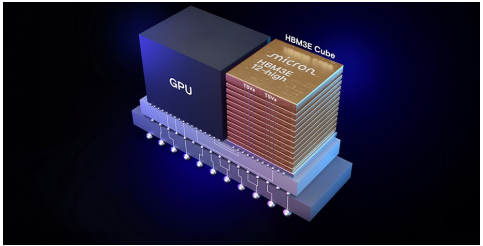
VRAM (HBM) là nơi chứa trọng số và bộ nhớ đệm KV (KV cache): dung lượng quyết định mô hình *có chạy được không*, băng thông quyết định *chạy nhanh đến đâu*.

Nguồn: NVIDIA, Hopper Architecture In-Depth (Bảng 3); NVIDIA không công bố băng thông các tầng trên

HBM thực tế: GPU và chồng nhớ chung một tấm đế



GPU AMD Fiji (2015) với 4 chồng HBM trên tấm interposer: kiến trúc đầu tiên đưa HBM vào GPU



Chồng HBM3E 12 lớp của Micron (ảnh dựng): các lớp DRAM nối nhau bằng TSV

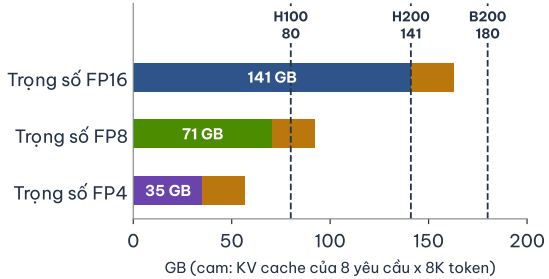
Đóng gói 2.5D đặt die GPU và các chồng HBM sát nhau trên tấm silicon, nên đường dẫn ngắn và có thể làm rất rộng. Thế hệ nay vẫn dùng ý tưởng này, nhưng với 5–8 chồng và 8–12 lớp mỗi chồng.

Nguồn: Wikimedia Commons (CC BY-SA 4.0); Micron HBM3E

Cần bao nhiêu bộ nhớ GPU cho một mô hình?

Công thức

1. **Trọng số** = số tham số $\times$ byte mỗi tham số
2. **Bộ nhớ đệm KV** mỗi token = $2\times$ số lớp $\times$ số đầu KV $\times$ kích thước đầu $\times$ byte
3. Chạy được khi: trọng số + bộ nhớ đệm KV + bộ nhớ trung gian $\leq$ dung lượng GPU



Llama-3-70B (70,55 tỉ tham số)

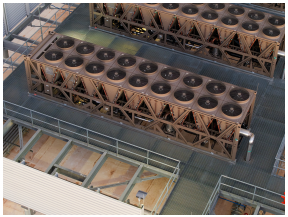
80 lớp, 8 đầu KV, kích thước đầu 128. FP16 **141 GB**; FP8 **70,6 GB**; FP4 **35,3 GB**. KV: **320 KiB mỗi token** (FP16), 2,7 GB cho 8K token.

FP16 không vừa một H200 (141,1 GB $>$ 141 GB). Hai H100 chia theo tensor còn dư khoảng 19 GB; một H200 chạy FP8 còn dư khoảng 70 GB.

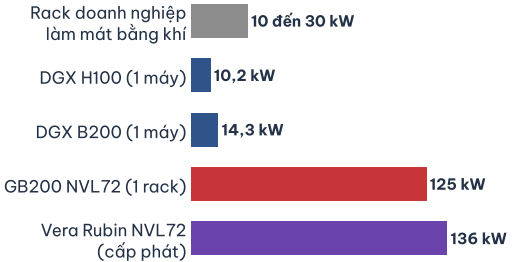
Nguồn: tính từ cấu hình Llama-3-70B; kiểm lại bằng mã Python

Trung tâm dữ liệu: điện và làm mát quyết định quy mô





Mái trung tâm dữ liệu với máy làm lạnh và máy phát điện; dàn quạt làm mát ở Arizona



Rack doanh nghiệp thường 10 đến 30 kW; rack GB200 NVL72 gấp 4 đến 12 lần nên phải làm mát bằng chất lỏng.

Nguồn: Rsparks3 (Wikimedia Commons, CC0); NVIDIA DGX và MaxLPS; Uptime Institute 2025

Phiên bản GPU và chi phí

Thế hệ GPU 2026, tên máy trên ba đám mây, giá thuê, chọn GPU theo loại tác vụ (task) và chiến lược giảm chi phí

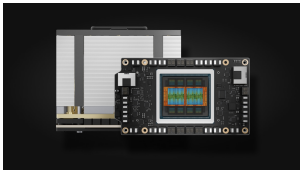
Thông số GPU trung tâm dữ liệu (số dense)

GPU	Bộ nhớ	Băng thông	FP8 dense	FP4 dense	TDP	NVLink	Tình trạng
A100 80GB	80 GB HBM2e	2,0 TB/s	—	—	400 W	600 GB/s	cũ
H100 SXM	80 GB HBM3	3,35 TB/s	1,98 PF	—	700 W	900 GB/s	phổ thông
H200	141 GB HBM3e	4,8 TB/s	1,98 PF	—	700 W	900 GB/s	phổ thông
B200	192 GB HBM3e	8 TB/s	4,5 PF	9 PF	≤1 200 W	1,8 TB/s	phổ thông
B300	288 GB HBM3e	8 TB/s	4,5 PF	13,5–15 PF	≤1 400 W	1,8 TB/s	đang giao
Rubin	288 GB HBM4	19,2–22 TB/s	17,5 PF	35 PF	n/a	3–3,6 TB/s	tăng tốc
AMD MI300X	192 GB HBM3	5,3 TB/s	2,61 PF	—	750 W	8 liên kết IF	hiện hành
AMD MI355X	288 GB HBM3E	8 TB/s	5 PF	10,1 PF	1 400 W	7 liên kết IF	đang giao

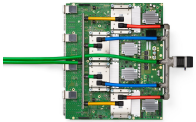
*PF = PFLOPS. Mọi số là **dense**; số NVIDIA quảng cáo thường là sparse (gấp đôi). B200: 192 GB là mức tối đa, máy thuê thực tế cung cấp 180 GB. Rubin: NVIDIA ghi băng thông 19,2 hoặc 22 TB/s tuỳ trang. B300 FP4: 13,5 (HGX) hoặc 15 (GB300 NVL72).*

Nguồn: trang sản phẩm NVIDIA và AMD, truy cập 01/10/2026

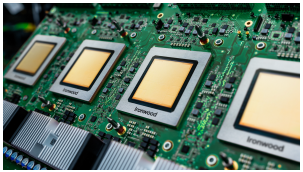
Hình ảnh thực tế của chip AI ngoài NVIDIA



AMD MI355X
module OAM, 288 GB HBM3E



Google TPU v4
4 chip, làm mát chất lỏng



Google TPU Ironwood
thế hệ 7, GA 03/2026



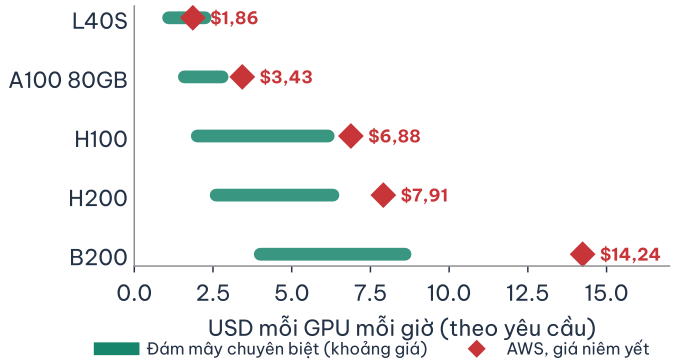
Siêu pod Ironwood: 9 216 TPU



Chip AWS (Annapurna Labs) và rack

Nguồn: AMD; Jouppi et al. (Wikimedia Commons, CC BY 4.0); Google Cloud; AWS

Giá thuê GPU: hai mặt bằng giá khác nhau



- Giá niêm yết trên đám mây lớn cao gấp **1,5 đến 3 lần** mặt bằng đám mây chuyên biệt cho cùng loại GPU
- Chênh lệch bù lại bằng dịch vụ quản lý, vùng khả dụng, SLA và tuân thủ
- Azure H100: 12,29 USD/GPU/giờ; GB200: 27,04 USD
- Bảng giá cũ trong tài liệu "H100 2,99–4,31 USD" thực chất là khoảng giá của nhà cung cấp chuyên biệt

Nguồn: AWS Price List API (25/09/2026); Azure Retail Prices API; trang giá Lambda, RunPod, CoreWeave, GMI (01/10/2026)

Cần bao nhiêu bộ nhớ GPU để suy luận và huấn luyện?

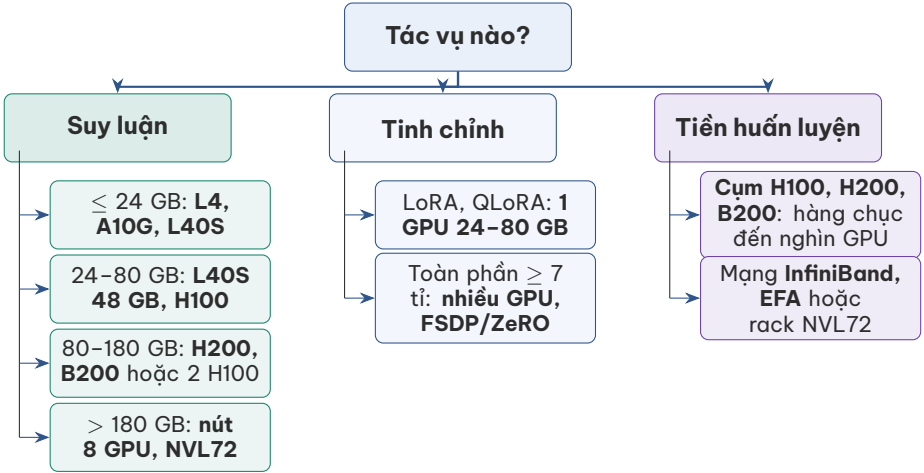
Mô hình	Suy luận FP16 (2 byte/tham số)	Suy luận FP8 (1 byte)	Suy luận FP4 (≈0,5 byte)	Tinh chỉnh toàn phần (16 byte/tham số)
7 tỉ	14 GB	7 GB	3,5 GB	112 GB
13 tỉ	26 GB	13 GB	6,5 GB	208 GB
70 tỉ	140 GB	70 GB	35 GB	1120 GB
405 tỉ	810 GB	405 GB	203 GB	6 480 GB

Vì sao tinh chỉnh toàn phần tốn 16 byte/tham số: trọng số 2 + gradient 2 + trạng thái của bộ tối ưu hoá (optimizer) Adam 12 (bản sao FP32, động lượng (momentum), phương sai).

Chưa tính: bộ nhớ đệm KV khi suy luận, kích hoạt (activation) khi huấn luyện. Các kỹ thuật như LoRA, chia trạng thái (ZeRO, FSDP) giảm đáng kể mức này.

Nguồn: Rajbhandari et al., ZeRO (2020) cho 16 byte/tham số; phần còn lại là phép tính trực tiếp

Cây quyết định chọn GPU



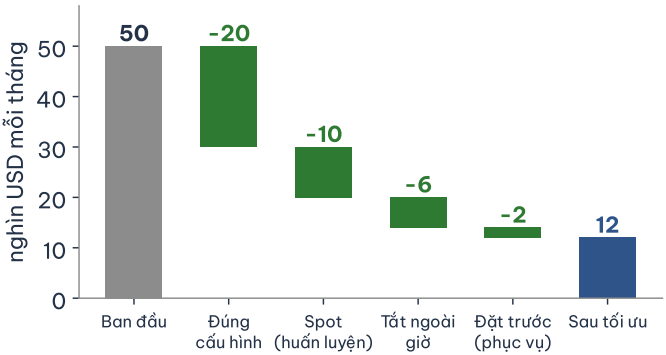
VRAM cần = trọng số + bộ nhớ đệm KV (suy luận), hoặc 16 byte/tham số (tinh chỉnh toàn phần)

Bốn lựa chọn mua: theo yêu cầu, spot, đặt trước, đặt chỗ GPU

Hình thức	Đặc điểm	Rủi ro	Dùng cho
Theo yêu cầu	Trả theo giờ, huỷ bất cứ lúc nào	Giá cao nhất; có thể hết GPU	Thử nghiệm, tải khó đoán
Spot (máy ảo dự phòng giá rẻ)	Rẻ hơn nhiều; bị thu hồi khi nhà cung cấp cần máy	Bị ngắt với thông báo ngắn (AWS: 2 phút)	Huấn luyện có điểm lưu, xử lý theo lô
Cam kết 1–3 năm	Giảm giá đổi lấy cam kết chi tiêu	Trả tiền cả khi không dùng	Phục vụ ổn định, tải nền
Đặt chỗ GPU	Giữ chỗ GPU cho khoảng thời gian cố định	Phải biết trước lịch	Đợt huấn luyện lớn, có ngày hẹn

Mức giảm điển hình (thay đổi theo GPU, vùng): spot 60–70%; cam kết 1 năm 30–40%; 3 năm 50–60%. **Quy tắc:** dưới 6 tháng dùng spot hoặc theo yêu cầu; trên 12 tháng cân nhắc cam kết.

Giảm hoá đơn GPU từ 50.000 xuống 12.000 USD mỗi tháng



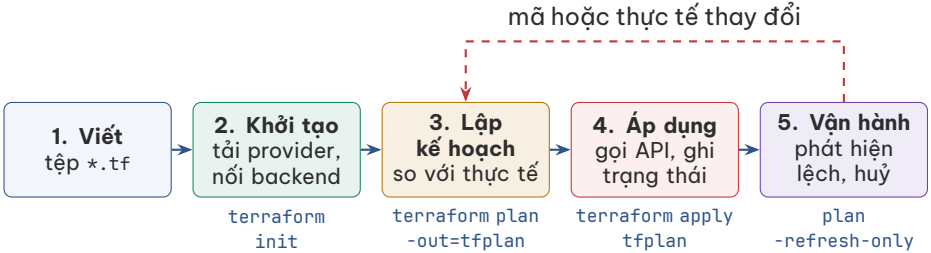
1. **Chọn đúng cấu hình** (right-sizing): đổi GPU quá lớn sang L40S hoặc A10G cho mô hình nhỏ
2. **Spot** cho huấn luyện và xử lý lô
3. **Tắt GPU ngoài giờ**: giảm về 0 bản sao đêm và cuối tuần
4. **Gói đặt trước** cho phần phục vụ ổn định

Tình huống minh hoạ để chia nhỏ các bước, không phải số đo từ một dự án cụ thể.

Terraform: hạ tầng dưới dạng mã

Vòng đời, trạng thái, mô-đun cho hạ tầng GPU, bẫy hạn mức và quy trình CI/CD có kiểm soát chi phí

Vòng đời làm việc với Terraform



Trạng thái (state) là bảng ánh xạ giữa khối `aws_instance.gpu` trong mã và máy `i-0abc…` thật trên AWS.

Đồ thị phụ thuộc suy ra từ các tham chiếu, không phải thứ tự tệp; nhánh độc lập được xử lý song song (mặc định 10 luồng).

Nguồn: HashiCorp, Terraform core workflow và Internals / Graph

Trạng thái từ xa: khoá bằng S3, không cần DynamoDB

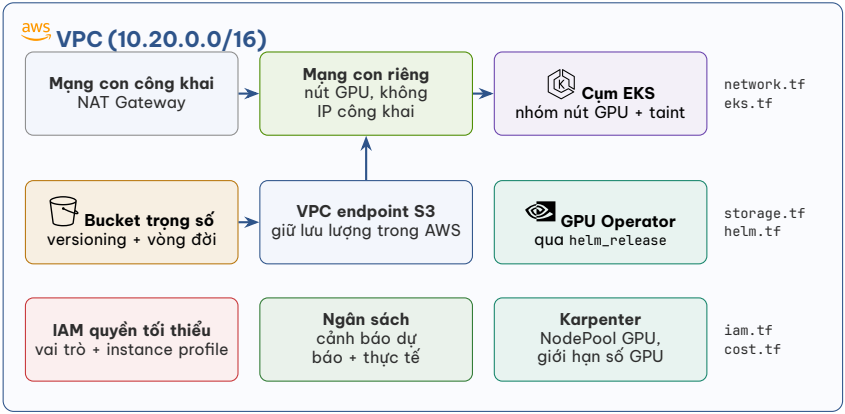
```
terraform {
  required_version = ">= 1.11"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 6.0" }
  }
  backend "s3" {
    bucket       = "aicb-tfstate-123456789012"
    key          = "track2/gpu-platform/terraform.tfstate"
    region       = "us-east-1"
    encrypt      = true
    use_lockfile = true
  }
}
```

Nguồn: HashiCorp, S3 backend; Terraform 1.10 (bản thử) và 1.11 (chính thức)

1. `use_lockfile` tạo tệp khoá `.tflock` ngay trong bucket: hai người không thể `apply` cùng lúc
2. Cách khoá bằng bảng DynamoDB đã **bị đánh dấu lỗi thời** từ Terraform 1.11
3. Bật *versioning* cho bucket để khôi phục trạng thái; `encrypt = true` để mã hoá

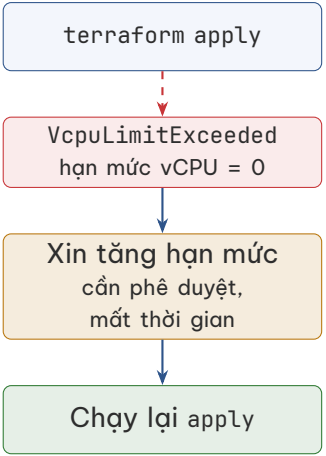
Lưu ý: Tệp trạng thái có thể chứa bí mật dạng rõ. Không đưa tệp này vào Git; giới hạn quyền đọc bucket.

Bản đồ cấu hình cho một nền tảng GPU trên AWS



Mỗi nhóm khối là một tệp `.tf`; chạy `terraform plan` trước, không `apply` cả bộ khi chưa đọc kế hoạch (có NAT Gateway, cụm EKS và tài nguyên tốn phí).

Bẫy đầu tiên: hạn mức GPU mới tạo bằng 0




```hcl
# G and VT On-Demand vCPUs
resource "aws_servicequotas_service_quota" "g" {
  service_code = "ec2"
  quota_code   = "L-DB2E81BA"
  value        = 32
}
```

Tài khoản mới có hạn mức vCPU **bằng 0** cho họ máy G, VT và P, cả loại theo yêu cầu lẫn spot. Yêu cầu tăng đi qua AWS xét duyệt, không tức thời: không để `apply` phụ thuộc vào bước này.

Nguồn: AWS, EC2 instance quotas; provider aws_servicequotas_service_quota

Nhóm nút GPU trên EKS

```
resource "aws_eks_node_group" "gpu" {
  cluster_name   = aws_eks_cluster.this.name
  node_role_arn  = aws_iam_role.eks_node.arn
  subnet_ids     = [aws_subnet.gpu_private.id]
  ami_type       = "AL2023_x86_64_NVIDIA"
  instance_types = ["g6e.2xlarge"]
  capacity_type  = "SPOT"
  scaling_config {
    min_size     = 0
    desired_size = 1
    max_size     = 4
  }
  taint {
    key    = "nvidia.com/gpu"
    value  = "true"
    effect = "NO_SCHEDULE"
  }
}
```

1. `AL2023_x86_64_NVIDIA`: image đã có driver và container toolkit. Image AL2 cũ đã ngừng phát hành từ 11/2025
2. `taint` giữ pod CPU khỏi nút đắt tiền
3. `min_size = 0`: nhóm thu về 0 khi không có pod GPU
4. `max_size = 4`: trần chi phí cứng

Lưu ý: AMI này **không** kèm device plugin của NVIDIA: phải cài riêng, ví dụ bằng GPU Operator.

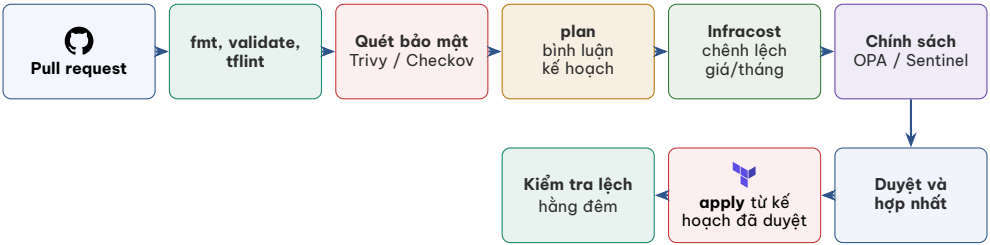
Karpenter: nhóm nút GPU tự co giãn, có trần chi phí

```yaml
apiVersion: karpenter.sh/v1
kind: NodePool
metadata: {name: gpu}
spec:
  template:
    spec:
      nodeClassRef:
        group: karpenter.k8s.aws
        kind: EC2NodeClass
        name: gpu
      requirements:
      - key: karpenter.k8s.aws/instance-family
        operator: In
        values: ["g6e"]
      taints:
      - key: nvidia.com/gpu
        effect: NoSchedule
  limits:
    nvidia.com/gpu: 8
```

1. Pod GPU ở trạng thái chờ → Karpenter chọn loại máy rẻ nhất thoả yêu cầu và tạo nút
2. `limits: nvidia.com/gpu: 8` là **trần số GPU** của pool, tức trần chi phí
3. Thêm khối `disruption` với `consolidationPolicy: WhenEmptyOrUnderutilized` để gộp và xoá nút trống
4. Cho phép cả `spot` và `on-demand` qua `karpenter.sh/capacity-type`

Chưa chạy thử trên cụm thật; cần thêm `EC2NodeClass`, vai trò IAM, hàng đợi gián đoạn. Nguồn: Karpenter docs (API karpenter.sh/v1).

CI/CD cho Terraform: kiểm tra, duyệt, rồi áp dụng



Chính sách mẫu cho GPU: cấm loại máy ngoài danh sách cho phép; cấm thiếu thẻ `CostCenter`; trần `max_size` của nhóm nút GPU.

Lưu ý Infracost: `aws_launch_template` không có giá riêng, nên chênh lệch chi phí chỉ hiện khi máy được khai báo qua `aws_instance`, nhóm Auto Scaling hoặc nhóm nút.

Dùng OIDC để CI nhận quyền tạm thời trên AWS, không lưu khoá tĩnh trong kho mã.

Terraform, OpenTofu, Pulumi, CDK: chọn công cụ nào?

Công cụ	Ngôn ngữ	Giấy phép	Ghi chú
Terraform 1.16	HCL	BSL 1.1 (IBM)	Phổ biến nhất; dùng nội bộ vẫn miễn phí
OpenTofu 1.13	HCL	MPL 2.0 (Linux Foundation)	Cùng hệ provider; có mã hoá state và `for_each` cho provider
Pulumi	Python, TypeScript, Go…	Apache 2.0 (CLI)	Viết bằng ngôn ngữ lập trình; phù hợp với nhóm đã quen Python
AWS CDK	TypeScript, Python…	Apache 2.0	Sinh CloudFormation; chỉ riêng AWS
Crossplane	YAML + Kubernetes API	Apache 2.0 (CNCF)	Vòng điều hoà liên tục thay vì plan/apply
CDKTF	—	—	Đã ngừng và lưu trữ 12/2025: không dùng cho dự án mới

Phiên bản ngày 01/10/2026. Trong tài liệu cũ, mục “CDK” nên hiểu là **AWS CDK**.

Nguồn: GitHub releases và tài liệu của từng dự án (01/10/2026)

Docker và Kubernetes cho AI

Đóng gói image GPU gọn, chạy trên cụm Kubernetes, cấp phát và mở rộng GPU theo nhu cầu

Image Docker nhiều giai đoạn (multi-stage)

```dockerfile
# Stage 1: build wheels (compiler available)
FROM nvidia/cuda:13.4.2-devel-ubuntu24.04 AS build
RUN apt-get update && apt-get install -y python3-pip
COPY requirements.txt .
RUN pip wheel -r requirements.txt -w /wheels

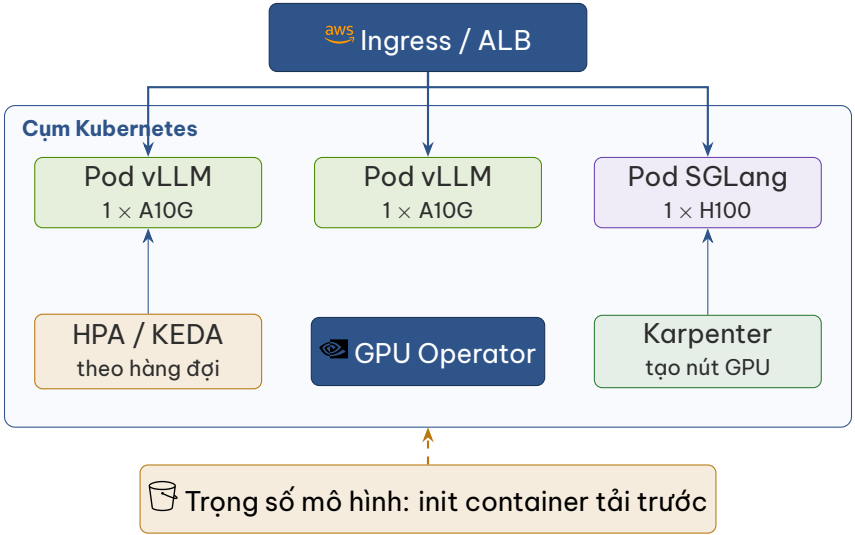
# Stage 2: runtime only
FROM nvidia/cuda:13.4.2-runtime-ubuntu24.04
RUN apt-get update && apt-get install -y python3-pip
COPY --from=build /wheels /wheels
RUN pip install --no-index --find-links=/wheels /wheels/*
COPY app/ /app/          # code AFTER dependencies
CMD ["python3", "-m", "app.serve"]
```

1. Image `devel` có trình biên dịch nên dung lượng lớn; chỉ dùng để dựng
2. Image `runtime` chỉ chứa thư viện CUDA cần khi chạy
3. Đặt lớp thư viện *trước* lớp mã: sửa mã không làm cài lại thư viện
4. `.dockerignore` loại tập dữ liệu và điểm lưu khỏi ngữ cảnh dựng

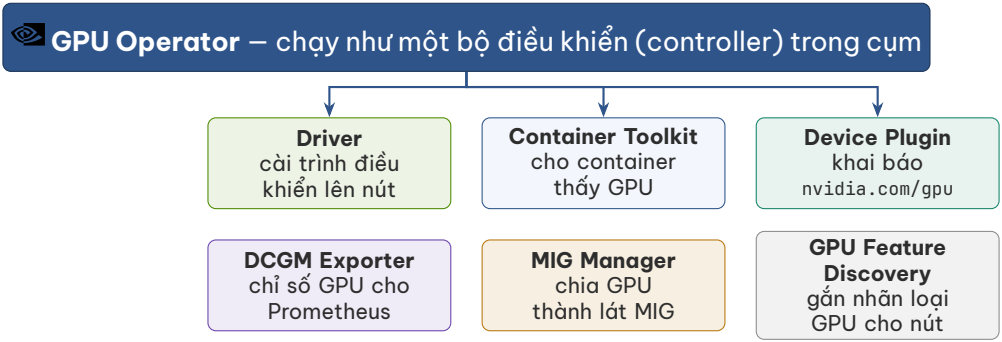
Tác dụng

Image nhỏ hơn thì kéo về nhanh hơn, **giảm độ trễ khởi động** (cold start) khi pod mới khởi động.

Kiến trúc Kubernetes cho phục vụ mô hình AI



NVIDIA GPU Operator: trọn chuỗi phần mềm GPU



Nhờ nhãn do GPU Feature Discovery gắn, một pod có thể chọn đúng loại GPU bằng `nodeSelector`; Karpenter hoặc Cluster Autoscaler tạo nút mới khi có pod chờ.

Chia sẻ một GPU: time-slicing, MPS, MIG

	Time-slicing	MPS	MIG
Cách chia	Luân phiên theo thời gian	Nhiều tiến trình dùng chung SM	Cắt GPU thành tối đa 7 lát phần cứng
Cách ly bộ nhớ	Không	Giới hạn mềm	**Có**, mỗi lát có bộ nhớ và SM riêng
GPU hỗ trợ	Hầu hết	Hầu hết	A100, H100, H200, B200 (dòng trung tâm dữ liệu)
Dùng khi	Thử nghiệm, tải nhẹ	Nhiều tiến trình nhỏ, cùng một người dùng	Nhiều mô hình nhỏ, khác người dùng, cần ổn định

Lưu ý: Mặc định Kubernetes cấp *nguyên* một GPU cho một pod. Chia sẻ là lựa chọn có chủ ý: cần cách ly thì dùng MIG; time-slicing không cách ly bộ nhớ, một pod hết bộ nhớ (OOM) có thể ảnh hưởng pod cùng GPU.

Thời gian một pod mới sẵn sàng nhận yêu cầu



khoảng 4 phút trước khi nhận yêu cầu đầu tiên

Rút ngắn từng đoạn

- nút: giữ sẵn vài nút ấm (warm pool) cho dịch vụ nhạy với độ trễ
- image: giảm kích thước, lưu đệm image trên nút
- trọng số: NVMe cục bộ, hệ tệp song song

Hệ quả khi thiết kế

- Tự động mở rộng mất khoảng vài phút để có hiệu lực: đặt ngưỡng sớm, giữ dư công suất
- Không giảm về 0 bản sao (scale-to-zero) với dịch vụ cần độ trễ thấp

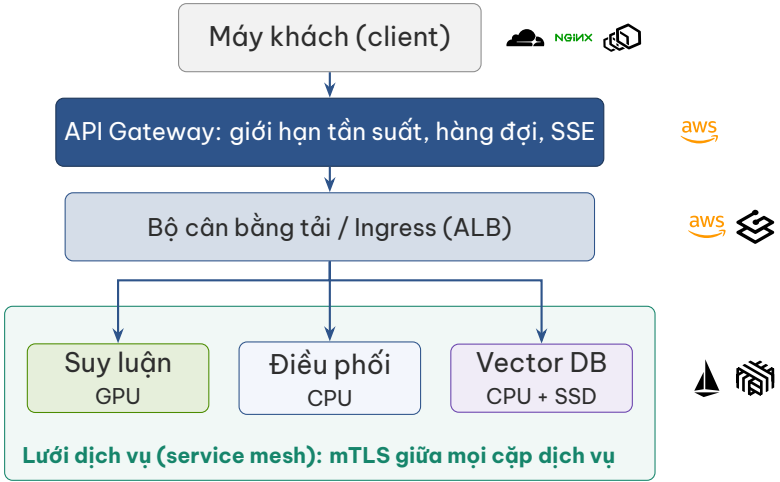
Nguồn: ví dụ minh hoạ về thứ tự và bậc độ lớn; thời gian thực phụ thuộc cấu hình, cần đo lại

07

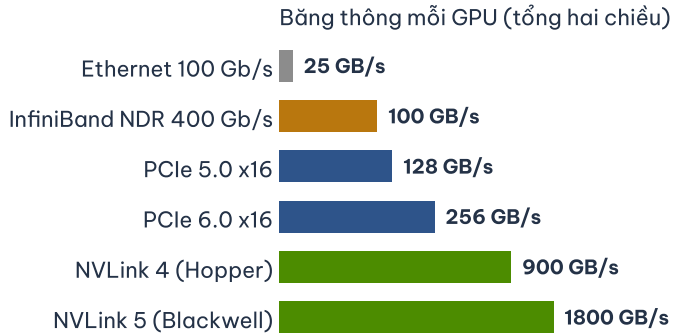
Mạng và lưu trữ cho AI

Đường đi của một yêu cầu, liên kết giữa các GPU, phân tầng lưu trữ và tốc độ nạp trọng số

Đường đi của một yêu cầu suy luận



Liên kết giữa các GPU: chênh nhau hàng chục lần



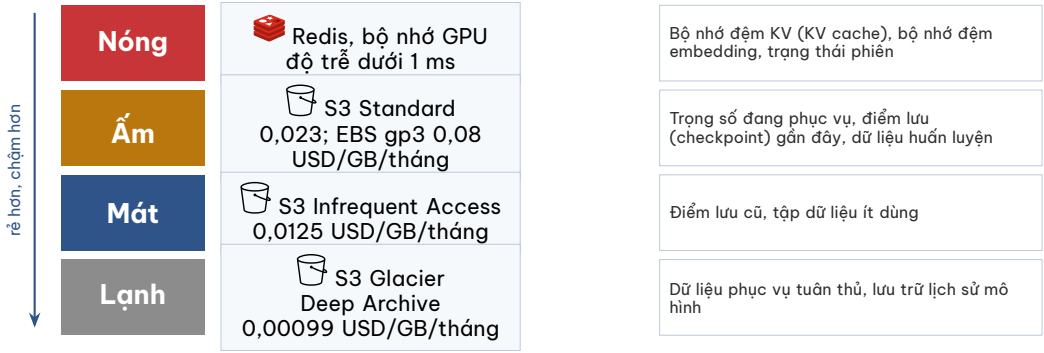
- **Trong một máy**: NVLink qua NVSwitch nối cả 8 GPU
- **Giữa các máy**: InfiniBand (400 Gb/s NDR, 800 Gb/s XDR) hoặc Ethernet RDMA; trên AWS là EFA (p5: 3 200 Gb/s mỗi máy)
- Lưu ý đơn vị: NVLink tính bằng GB/s, mạng bằng Gb/s; 900 GB/s = 7 200 Gb/s

Hệ quả

Chia mô hình theo tensor chỉ nên nằm **trong một máy NVLink**.

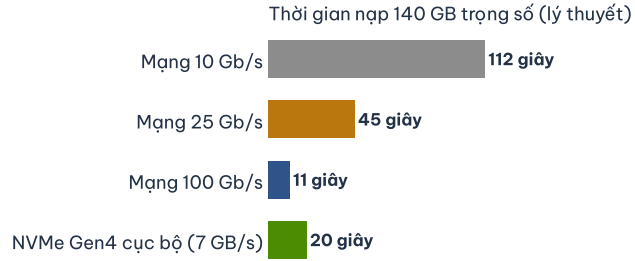
Nguồn: NVIDIA (NVLink 4/5), PCI-SIG (PCIe 5.0), AWS EFA; số liệu cho một chiều hoặc tổng hai chiều được ghi rõ trong hình

Lưu trữ phân tầng: nóng, ấm, mát, lạnh



Giá S3 theo vùng us-east-1 (tầng đầu tiên); kiểm tra lại trên trang giá của AWS trước khi ước tính.

Nạp trọng số: điểm nghẽn của độ trễ khởi động



- Mô hình 70 tỉ tham số ở FP16 nặng khoảng **140 GB**
- Pod mới phải nạp xong trọng số mới nhận được yêu cầu: thời gian này cộng thẳng vào độ trễ khởi động
- Cách rút ngắn: ổ NVMe cục bộ, hệ tệp song song (FSx for Lustre), nạp trước bằng init container

Nguồn: tính từ băng thông lý thuyết của đường truyền, chưa gồm chi phí giao thức; thực tế chậm hơn

Hạ tầng cho agent: 8 lớp

Từ tài nguyên tính toán đến quản lý bí mật — mỗi lớp một bài toán riêng

08

Tám lớp hạ tầng của một agent ở môi trường vận hành

8 Bí mật	Vault · Secrets Manager · cờ tính năng (feature flag)
7 Quan sát	OpenTelemetry · Jaeger · Prometheus · Grafana
6 Mạng	API Gateway · gRPC nội bộ · HTTP + SSE (truyền tải MCP)
5 Lưu trữ	PostgreSQL + pgvector · Redis · S3
4 Bộ nhớ đệm	L1 trong tiến trình · L2 Redis · L3 bộ nhớ đệm embedding
3 Hàng đợi	Redis Streams · Kafka · RabbitMQ
2 Điều phối	LangGraph · CrewAI · trên pod CPU
1 Tính toán	GPU (suy luận) · CPU (điều phối) · Lambda (công cụ)

Nguyên tắc thiết kế: agent *không lưu trạng thái* (stateless); trạng thái được chuyển sang Redis hoặc Postgres để mở rộng ngang.

Bộ phục vụ mô hình

Bộ công nghệ (stack) phục vụ mô hình: vLLM, SGLang và các lựa chọn khác: cách chúng dùng bộ nhớ GPU, cách triển khai và cách đo

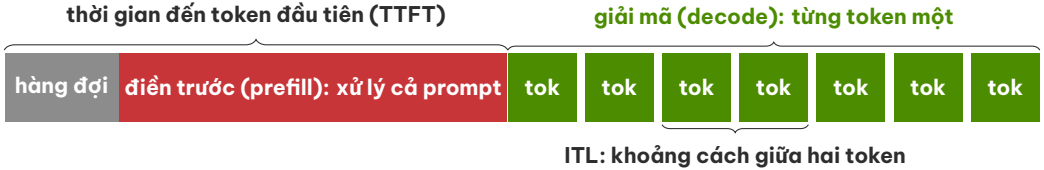
Bức tranh bộ phục vụ mô hình năm 2026

Lớp điều phối nhiều nút	NVIDIA Dynamo llm-d KServe Ray Serve
Công cụ suy luận (engine)	vLLM SGLang LMDeploy TensorRT-LLM
Chạy cục bộ	Ollama (hệ sinh thái llama.cpp)
Đang bảo trì	Text Generation Inference (TGI): repo lưu trữ 03/2026

Hai lớp: **engine** chạy mô hình trên một nhóm GPU; **lớp điều phối** định tuyến yêu cầu, tách giai đoạn và co giãn qua nhiều nút.

Nguồn: README các dự án, PyPI/GitHub (01/10/2026)

Suy luận gồm hai giai đoạn với hai nút thắt khác nhau



Prefill: giới hạn bởi tính toán

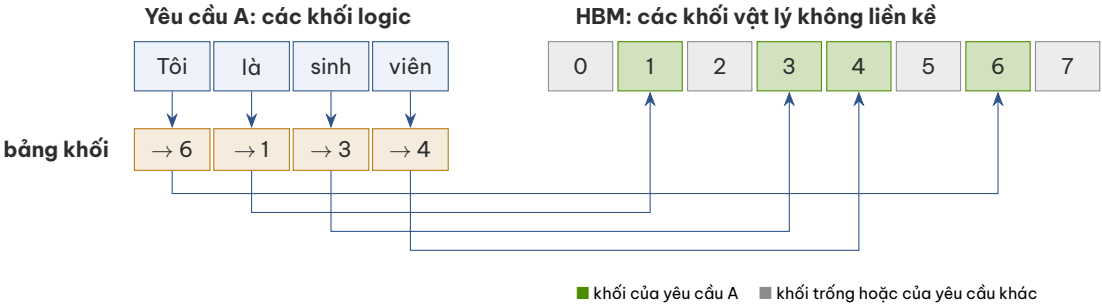
Mọi token của prompt được xử lý song song: phép nhân ma trận–ma trận, GPU gần như bão hoà. Quyết định **TTFT**.

Decode: giới hạn bởi băng thông

Mỗi bước sinh một token, phải đọc lại toàn bộ trọng số và bộ nhớ đệm KV: phép nhân ma trận–vector. Quyết định **ITL**.

Nguồn: NVIDIA, Mastering LLM Techniques: Inference Optimization; định nghĩa chỉ số theo tài liệu vLLM

PagedAttention: chia bộ nhớ đệm KV thành khối

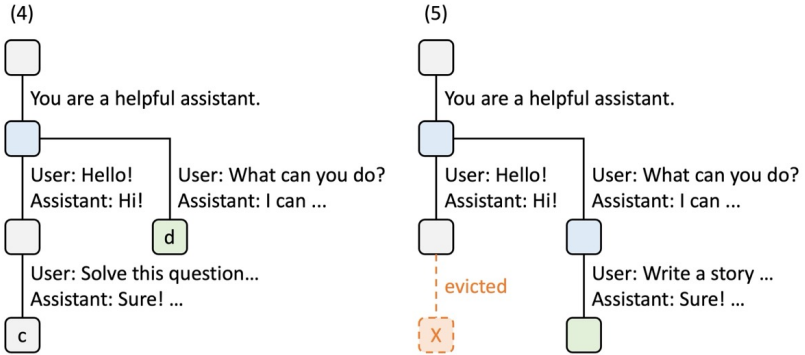


Ý tưởng: giống bộ nhớ ảo của hệ điều hành. Bộ nhớ đệm KV được cắt thành khối nhỏ, cấp khi cần qua bảng khối, nên gần như không còn phân mảnh.

Kết quả (bài báo, 2023): thông lượng gấp 2 đến 4 lần so với FasterTransformer và Orca ở cùng độ trễ. Khối còn dùng chung được giữa các yêu cầu có chung tiền tố.

Nguồn: Kwon et al., arXiv:2309.06180; sơ đồ vẽ minh hoạ

RadixAttention của SGLang: dùng lại tiền tố bằng cây



1. Mọi prompt đã xử lý được lưu thành **cây tiền tố (radix tree)**: nút gốc chung là "You are a helpful assistant", hai nhánh là hai hội thoại khác nhau.
2. Yêu cầu mới tìm tiền tố dài nhất đã có trong cây và **bỏ qua prefill** đoạn đó.
3. Khi đầy bộ nhớ, nút lá ít dùng gần đây nhất bị loại (hình phải, nhánh "evicted").
Hợp với: hội thoại nhiều lượt và agent lặp lại cùng chỉ dẫn hệ thống.

Nguồn: LMSYS, SGLang blog (01/2024), panel 4 và 5; paper arXiv:2312.07104

So sánh các engine suy luận

Engine	Kỹ thuật chính	Điểm mạnh	Hợp với
vLLM 0.30	PagedAttention, gom lô liên tục, lưu đệm tiền tố	Hệ sinh thái rộng nhất, API tương thích OpenAI, nhiều định dạng lượng tử hoá	Mặc định cho hầu hết dịch vụ
SGLang 0.5	RadixAttention, đầu ra có cấu trúc	Mạnh khi nhiều yêu cầu chung tiền tố	Agent, hội thoại nhiều lượt
LMDeploy 0.18	TurboMind (C++)	Tối ưu cho mô hình lượng tử hoá	Triển khai nhạy độ trễ
TensorRT-LLM 1.2	Kernel tối ưu của NVIDIA	Hiệu năng cao trên GPU NVIDIA, cần biên dịch engine	Quy mô lớn, GPU cố định
Ollama 0.35	Hệ sinh thái llama.cpp	Một lệnh chạy, đổi mô hình dễ	Máy cá nhân, phát triển
TGI 3.3	—	Đang ở chế độ bảo trì, repo lưu trữ	Hệ thống cũ, không chọn mới

Phiên bản ngày 01/10/2026; ra bản mới gần như hằng tuần. Nguồn: README từng dự án.

Khởi chạy vLLM và SGLang

vLLM (từ 0.30)

```
vllm serve Qwen/Qwen2.5-7B-Instruct \
  --tensor-parallel-size 1 \
  --max-model-len 8192

# default --gpu-memory-utilization = 0.92
# lower it only when sharing the GPU
#
```

- Cách cũ `python -m vllm.entrypoints.openai.api_server` đã bị đánh dấu lỗi thời
- `--gpu-memory-utilization`: tỉ lệ bộ nhớ một tiến trình được dùng; **tăng** để có thêm KV, **giảm** khi hai tiến trình chung GPU

SGLang (0.5.x)

```
python3 -m sglang.launch_server \
  --model-path Qwen/Qwen2.5-7B-Instruct \
  --port 30000 \
  --tp 1

# newer CLI: sglang serve ...
# 0.5.20+ requires CUDA 13
```

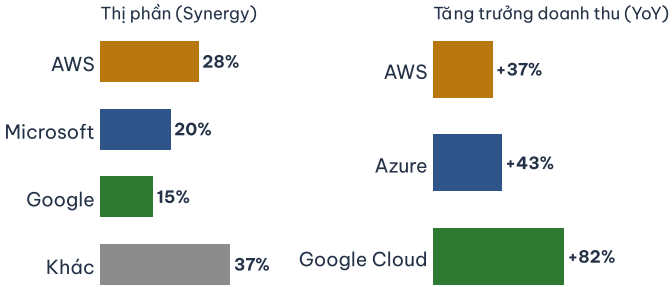
- `--max-model-len`: đặt theo độ dài thực tế; không đặt 128K khi 99% yêu cầu dưới 4K
- `--tp` / `--tensor-parallel-size`: số GPU chia mô hình theo tensor
- Cả hai mở API tương thích OpenAI

Nguồn: vLLM CLI docs và README; SGLang server arguments (01/10/2026)

Bức tranh đám mây và hạ tầng AI 2026

Quy mô thị trường, dòng vốn đầu tư, ai thuê ai tự xây và bốn xu hướng định hình ngành

Thị trường đám mây quý 2/2026



143,4 tỉ USD

doanh thu hạ tầng đám mây quý 2/2026, tăng 43% so với cùng kỳ

Mức tăng cao nhất trong 8 năm; dịch vụ đám mây AI sinh tạo (GenAI) tăng 165%.

Thị phần: IaaS, PaaS, đám mây riêng (Synergy). Tăng trưởng: doanh thu mảng đám mây do từng hãng công bố; Google Cloud gồm cả Workspace.

Nguồn: Synergy (30/07/2026); Amazon, Microsoft, Alphabet: báo cáo quý 2/2026

Ai thuê năng lực tính toán, ai tự xây?

Đa nền tảng: Anthropic

- **AWS**: tới 5 GW, hơn 100 tỉ USD trong 10 năm (04/2026); Project Rainier dùng hơn 1 triệu chip Trainium2
- **Google**: tới 1 triệu TPU
- **SpaceX/xAI**: hơn 300 MW, hơn 220 000 GPU tại Colossus 1 (05/2026)

Quy mô riêng và liên doanh

- **OpenAI**: Stargate với Oracle và SoftBank, gần 7 GW kế hoạch, hơn 400 tỉ USD trong 3 năm (công bố 09/2025)
- **Google**: dùng chính TPU Ironwood, siêu pod 9 216 chip
- **Meta**: tự xây Hyperion (mục tiêu 5 GW), thêm chip riêng
- **xAI**: là công ty con của SpaceX từ 02/2026

Không phòng thí nghiệm lớn nào phụ thuộc một nhà cung cấp hay một loại chip.

Nguồn: Anthropic (04 và 05/2026); Google Cloud; OpenAI (09/2025); Louisiana ED (07/2026); SpaceX S-1

Bốn xu hướng định hình hạ tầng AI

1. **Đa đám mây, đa loại chip** là chuẩn mới: không phòng thí nghiệm lớn nào chỉ dựa vào một nhà cung cấp.
2. **Chip riêng cạnh tranh với NVIDIA:** Google TPU, AWS Trainium, Meta MTIA, AMD Instinct tăng tốc.
3. **Thuê chéo năng lực tính toán:** đối thủ cho nhau thuê khi dư năng lực, như hợp đồng Anthropic–SpaceX.
4. **Điện là nút thắt mới:** đất, điện, làm mát khan hiếm hơn chip; dự án tính bằng gigawatt.

LAB #16

Mục tiêu: Dựng môi trường AI trên đám mây: mạng riêng, máy GPU, bộ phục vụ vLLM hoặc SGLang, ngân sách và ước tính chi phí

Deliverable: Môi trường chạy được, tệp ước tính chi phí, ảnh chụp bảng theo dõi chi phí và điểm cuối agent hoạt động

Thời gian: 2 giờ

Tổng kết

1. Chọn nhà cung cấp theo khối lượng, ngân sách, tuân thủ, độ trễ: đám mây lớn cho dịch vụ cần SLA, GPU chuyên biệt cho huấn luyện, đám mây trong nước cho dữ liệu nội địa.

2. Chọn GPU theo **bộ nhớ trước**: trọng số cộng bộ nhớ đệm KV phải vừa VRAM. Giải mã bị giới hạn bởi băng thông HBM, không phải FLOPs.

3. Giảm chi phí: đúng cấu hình, spot có điểm lưu, tắt GPU ngoài giờ. Dùng Terraform với trần số GPU, ngân sách và thẻ ngay từ đầu.

4. Phục vụ mô hình là bài toán bộ nhớ và hàng đợi: vLLM là mặc định, SGLang cho tiền tố chung; đo TTFT, ITL, thông lượng.

Hỏi & Đáp



Cảm ơn!

AICB-P2T2 · Ngày 16 · Hạ tầng đám mây cho AI

Giảng viên Nguyễn Hải Dương · VinUniversity